

Hastane Randevu ve Hasta Öncelik Sistemi

BMT210 Veri Yapıları Dersi — Dönem Projesi
Gazi Üniversitesi · Teknoloji Fakültesi · Bilgisayar Mühendisliği

Muhammet Eren Demirtaş - Onur Olmuş - Hayri Topaloğlu

Problem

Binlerce hasta kaydında anlık, verimli arama ve acil önceliklendirme ihtiyacı

Yaklaşım

Hash Table, Min-Heap, Queue, Stack ve N-ary Tree ile modüler sistem mimarisi

Sonuç

Bağlı listelerle kıyaslandığında 10.000 kayıttaki **2115x** hız artışı elde edildi

Teknoloji

C (POSIX Socket backend) + JSON/HTTP + HTML/CSS/JS Web Arayüzü — sıfır harici kütüphane

SİSTEM MİMARİSİ

Problem ve Sistem Mimarisi

Hastane bilgi sistemlerinde en kritik zorluklar dört temel başlık altında toplanmaktadır. Bu projenin amacı, her bir sorunu en uygun veri yapısıyla çözmek ve ölçülebilir performans kazanımları elde etmektir.



Anlık Hasta Arama

Binlerce kayıt içinde TC kimlik numarasına göre **O(1)** ortalama karmaşıklıkta anlık arama. Bağlı liste ile aynı işlem **O(n)** zaman alır.



Adil Randevu Sırası

Bağlı liste tabanlı Queue (FIFO) ile her iki uçta **O(1)** işlem. Sabit dizi tabanlı çözümlerde dequeue **O(n)** kaydırma gerektirir.

MİMARİ BİLEŞENLER

C Backend (POSIX Socket)

Tüm veri yapıları C dilinde, harici kütüphane kullanılmadan sıfırdan implement edilmiştir. POSIX Socket katmanı üzerinden HTTP benzeri istekleri karşılar ve JSON formatında yanıt üretir.

Hash Table	Min-Heap
Queue	Stack
N-ary Tree	



Acil Triağ Önceliği

Min-Heap yapısıyla en kritik hasta daima kuyruğun başındadır. **O(log n)** ekleme ve çıkarma ile gerçek zamanlı önceliklendirme sağlanır.



İşlem Geri Alma

Stack (LIFO) ile son işlem her zaman en üstte; push ve pop **O(1)**. Undo mekanizması ekle/sil işlemlerini tam tersine çevirir.

Web Arayüzü (HTML/CSS/JS)

Tarayıcı tabanlı dashboard, arka uçta **JSON/HTTP** protokolü üzerinden haberleşir. Gerçek zamanlı triağ renk kodlaması, kuyruk görünümü ve yük faktörü izleme ekranları sunar.

İletişim Katmanı: **C Backend** ↔ **JSON/HTTP** ↔ **Web Arayüzü**
Tüm veri akışı bu üçlü mimari üzerinden yönetilir.

VERİ YAPILARINA GENEL BAKIŞ

5 Veri Yapısının Karmaşıklık Özeti

Her modülde veri yapısı seçimi, işlemin karmaşıklık gereksinimine ve erişim örüntüsüne göre yapılmıştır. Aşağıdaki tablo tüm modülleri tek bakışta karşılaştırmaktadır.

Modül	Veri Yapısı	Temel İşlem	Karmaşıklık
Hasta Kaydı	Hash Table	TC kimlik no ile arama	O(1) ort.
Acil Triağ	Min-Heap	En kritik hastayı bul	O(log n)
Randevu Yönetimi	Queue (Linked List)	FIFO sıralama	O(1)
İşlem Geçmişi	Stack (Linked List)	Son işlemi geri al	O(1)
Poliklinik Yapısı	N-ary Tree	Hiyerarşik organizasyon	O(n) DFS

Temel Prensipte: Her modül için veri yapısı, işlemin karmaşıklık gereksinimine göre seçilmiştir. Yanlış veri yapısı seçimi aynı işlemi binlerce kat yavaşlatabilir.

Detaylı Modül Analizi ve Performans Sonuçları

Hash Table · Min-Heap · Queue · Stack · N-ary Tree — Her modülün algoritması, karmaşıklığı ve ölçülen performans kazanımları

MODÜL 1

Hash Table: Hasta Kaydı Sistemi

Algoritma Detayı

TC kimlik numarası üzerinde **Polynomial Rolling Hash** fonksiyonu uygulanır:

$$h(tc) = \sum (tc[i] \times 31^i) \bmod 1009$$

- 1009 asal bucket boyutu** — asal sayı kullanımı hash değerlerinin homojen dağılmasını garanti eder; yük faktörü düşük kalır
- Çakışma çözümü: Chaining** — her bucket bir bağlı liste zinciri tutar; aynı hash değerine düşen kayıtlar zincire eklenir
- Yük faktörü izleme** — dashboard üzerinden canlı olarak görselleştirilir; kritik eşiğe yaklaştığında uyarı verilir
- Ortalama durum **O(1)** arama; en kötü durum (tüm çakışma) **O(n)** — ancak iyi hash fonksiyonu ile bu durum son derece nadirdir

MODÜL 2

Min-Heap: Acil Triage Sistemi

Heap Yapısı ve İşlemler

Min-Heap, tam ikili ağaç özelliğini koruyan ve her düğümün çocuklarından küçük/eşit olduğu bir yapıdır. Aciliyet skoru **1= en kritik** olacak şekilde tanımlanmıştır; kök her zaman en kritik hastayı tutar.

Sift-Up (Ekleme)

Yeni hasta yaprağa eklenir → ebeveynden küçük/eşit yer değiştirilir → kök veya doğru konuma ulaşana dek tekrar. **O(log n)**

Sift-Down (Çıkarma)

Kök çıkarılır → son yaprak köke taşınır → çocuklardan büyükse yer değiştirilir → doğru konuma kadar. **O(log n)**

⚠ Sıralı dizide her ekleme **O(n)** kaydırma gerektirir — Min-Heap'te yalnızca **O(log n)** sift-up yeterlidir.

Arama Süresi Karşılaştırması (ms)

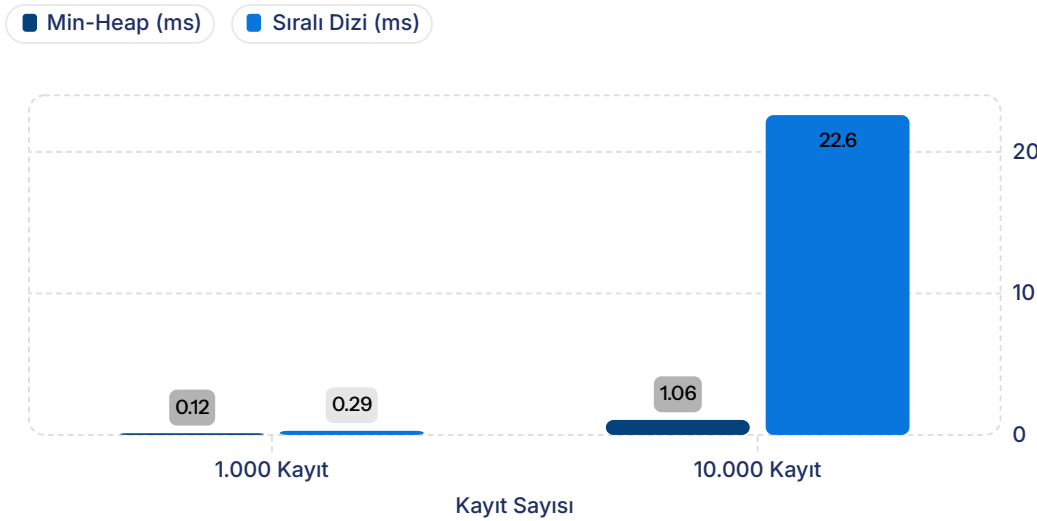


2115x

Daha Hızlı

10.000 kayıttan Hash Table vs Bağlı Liste

Ekleme Süresi Karşılaştırması (ms)



21x

Daha Hızlı

10.000 kayıttan Min-Heap vs Sıralı Dizi

MODÜLLER 3 & 4

Queue ve Stack: Randevu Yönetimi ve İşlem Geçmişi

Queue — Randevu Yönetimi (FIFO)

Bağlı liste tabanlı Queue, iki işaretçi (on/arka) sayesinde her iki uçta da sabit zamanlı işlem sunar. İlk gelen hasta ilk hizmet alır prensibi sağlanır.

- Enqueue (Arka pointer):** Yeni randevu kuyruğun sonuna eklenir → **O(1)**
- Dequeue (Ön pointer):** En eski randevu kuyruğun başından çıkarılır → **O(1)**
- Sabit boyutlu dizide dequeue, tüm elemanları bir sola kaydırır → **O(n)** — bağlı liste bu sorunu tamamen ortadan kaldırır
- Bellek dinamik olarak yönetilir; maksimum boyut sınırı yoktur

Stack — İşlem Geçmişi (LIFO)

Bağlı liste tabanlı Stack, işlem geçmişini yönetmek ve tam undo mekanizması sunmak için kullanılır. En son yapılan işlem her zaman en tepededir.

- Push O(1):** İşlem kaydı stack'in tepesine eklenir; hiçbir kaydırma yapılmaz
- Pop O(1):** Son işlem tepeden alınır; geri alma anında gerçekleşir
- Undo Mantığı:** HASTA_EKLE kaydedildiyse → geri almada sil; HASTA_SIL kaydedildiyse → geri almada yeniden ekle
- Queue ile son işleme erişim **O(n)** gerektirir — Stack ile **O(1)**

MODÜL 5

N-ary Tree: Poliklinik Hiyerarşisi

Ağaç Yapısı

Hastane organizasyonu doğal bir hiyerarşik yapıya sahiptir. N-ary Tree, her düğümün keyfi sayıda çocuk tutabilmesine olanak tanır; bu da poliklinik/bölüm ilişkilerini doğrudan modellemek için idealdir.

1 Hastane (Kök)

Dahiliye → Kardiyoloji, Nöroloji, Gastro

2 Cerrahi

Genel Cerrahi, Ortopedi

3 Acil

Travma, Kardiyak Acil

Neden N-ary Tree? Alternatif Karşılaştırması

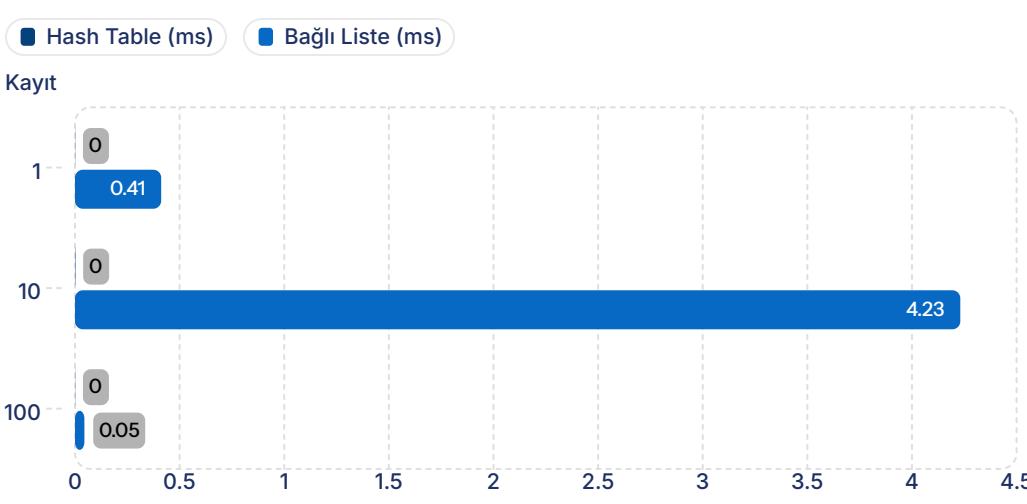
Yaklaşım	Alt Birim Bulma	Hiyerarşi
N-ary Tree ✓	O(k) doğrudan erişim	Doğal, örtük
Düz Liste ✗	O(n) tüm liste taranır	Dolaylı, zorlanmış
Hash Map ✗	O(n) ayrı tarama	Hiyerarşi yoktur
Graf ✗	O(V+E) döngü kontrolü	Aşırı karmaşık

✓ DFS (Derinlik Öncelikli Arama) ile tüm poliklinikler **O(n)** karmaşıklıkta ziyaret edilir; hem listeleme hem de arama bu yapı üzerinden gerçekleştirilir.

GENEL PERFORMANS

Tüm Modüllerin Performans Karşılaştırması

Arama: Hash Table vs Bağlı Liste

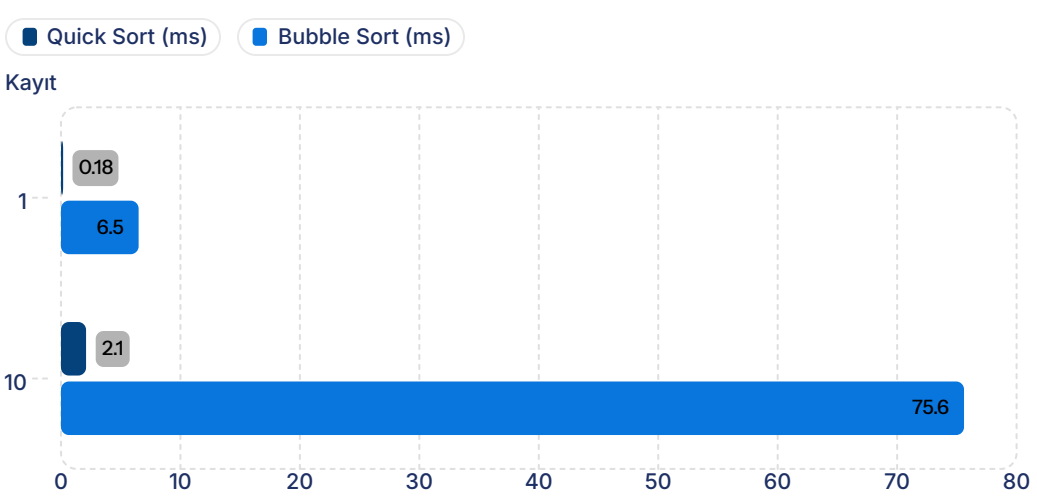


2115x

Hash Table Kazandı

10.000 kayıttan Hash Table vs Bağlı Liste araması

Sıralama: Quick Sort vs Bubble Sort



21x

Min-Heap Kazandı

10.000 kayıttan Min-Heap vs Sıralı Dizi eklemesi

36x

Quick Sort Kazandı

10.000 kayıttan Quick Sort vs Bubble Sort sıralaması

TEST SONUÇLARI & SONUÇ

Test Özeti ve Temel Çıkarımlar

Fonksiyonel ve Sınır Durum Testleri

- ✓ 14 işlevsel test — tümü başarılı
- ✓ 6 sınır durum testi — tümü başarılı
- ✓ 3 veri seti boyutu test edildi: 100 / 1.000 / 10.000 kayıt
- ✓ Hash Table yük faktörü canlı izleme — dashboard üzerinden görselleştirildi
- ✓ Triage renk kodlaması: 🔴 Kritik / 🟡 Orta / 🟢 Stabil

Toplam **20 test senaryosu** uygulanmış olup hiçbirinde başarısızlık kaydedilmemiştir. Sınır durumları arasında boş kuyruk, dolu heap ve yinelenen TC kimlik numarası testleri yer almaktadır.

Dashboard İstatistikleri

Toplam Hasta

10.000 kayıt yüklendi ve test edildi

Randevu Kuyruğu

FIFO Queue canlı görünümü

Triage Bekleyen

Min-Heap öncelik sırası

Bugünkü İşlem

Stack undo geçmişi ile izlendi

Sonuç: Temel Çıkarımlar

1

Bilinçli Veri Yapısı Seçimi

Her modül için karmaşıklık gereksinimi önce analiz edilmiş, ardından en uygun veri yapısı seçilmiştir. Tek tip bir yapı kullanmak yerine probleme özgü çözümler benimsenmiştir.

2

Ölçülebilir Performans Kanıtı

Performans farkları 100, 1.000 ve 10.000 kayıt boyutlarında tekrarlanabilir testlerle doğrulanmıştır. Teorik karmaşıklık analizi pratikte de geçerliliğini kanıtlamıştır.

3

Sıfır Harici Bağımlılık

Sistem, Hash Table'dan N-ary Tree'ye kadar tüm veri yapılarını C ile sıfırdan inşa etmiştir. Bu yaklaşım derin algoritmik anlayışı ve düşük seviye bellek yönetimi yetkinliğini ortaya koyar.

4

2000 Kata Kadar Kazanım

Doğru veri yapısı seçimi, aynı problemi çözen naif uygulamaya göre performansı **2000 kata kadar** artırabilir. Bu fark, gerçek dünya sistemlerinde yanıt süresi ve ölçeklenebilirlik açısından kritiktir.

✓ **Hastane Randevu ve Hasta Öncelik Sistemi** — BMT210 Veri Yapıları Dönem Projesi · Gazi Üniversitesi Teknoloji Fakültesi · Bilgisayar Mühendisliği
Muhammet Eren Demirtaş

Made with GAMMA